Validación técnica de Odoo 20 Community mediante API JSON-2

Pruebas reproducibles de integración para el proyecto de Aplicaciones Móviles

Documento de evidencia técnica | 25 de octubre de 2026 | Versión 1.1

Alcance: creación, confirmación, consulta y filtrado de pedidos sale.order. No constituye la implementación del proyecto estudiantil.

1. Propósito y límites

Registrar fielmente el procedimiento ejecutado para verificar la viabilidad básica de integrar Odoo 20 Community con una aplicación externa mediante JSON-2. Se emplearon Postman, la documentación Runtime Doc de Odoo y consultas de diagnóstico de solo lectura en PostgreSQL. La implementación de Flutter/Android, la API REST de ASP.NET Core, Clean Architecture, autenticación y carrito corresponde a los estudiantes.

2. Entorno y datos de prueba

Componente	Configuración observada
Sistema	Windows, Docker Desktop con WSL 2
Contenedores	odoo:20.0 y postgres:16
Carpeta de trabajo	D:\Odoo20-Docker
Base de datos	odoo20_integradores
Odoo web	http://localhost:8069
Documentación API	http://localhost:8069/doc
Cliente de API	Postman, Bearer Token
Productos	ID 7 (PROD-TEST-001), ID 8 (PROD-TEST-002)
Cliente	Consumidor final, res.partner ID 6
Pedido generado	sale.order ID 1, S00001

Seguridad: la API Key no se incluye en este documento. Debe introducirse localmente en Postman, sin publicarla ni incorporarla a la aplicación móvil.

2.1. Fundamentos mínimos para interpretar las pruebas

Esta sección explica los términos que se emplean en las solicitudes y respuestas. Los identificadores y resultados mencionados corresponden a nuestra instalación de prueba; no son valores universales.

1. API REST y JSON-2. Una API permite que otra aplicación solicite operaciones a Odoo. JSON-2 es la interfaz HTTP de Odoo utilizada en esta validación: las llamadas se envían con POST a rutas como

/json/2/sale.order/create, incluso cuando la operación consiste en consultar. No debe suponerse que todas las rutas siguen las convenciones de una API REST basada en GET, POST, PUT y DELETE.

2. Modelo, registro y campo. Un modelo representa un tipo de información: sale.order corresponde a pedidos de venta y res.partner a contactos. Un registro es una instancia concreta, por ejemplo el pedido S00001. Un campo es un dato de ese registro, como partner_id o state.

3. API Key y cabeceras HTTP. La API Key es la credencial que autoriza las solicitudes y se envía como Bearer Token en la cabecera Authorization. X-Odoo-Database indica la base de datos y Content-Type: application/json identifica el formato del cuerpo. No se deben publicar credenciales ni incluirlas directamente en la aplicación móvil.

4. Métodos create, search_read y action_confirm. create registra una nueva cotización; search_read consulta registros y devuelve los campos solicitados; action_confirm confirma una cotización existente. Son operaciones diferentes: crear una cotización no significa que ya esté confirmada o pagada.

5. sale.order y sale.order.line. sale.order representa la cabecera del pedido (cliente, referencia, estado, importes). sale.order.line representa cada producto o línea del pedido (producto, cantidad, precio). El campo order_line relaciona la cabecera con sus líneas; en la creación se utilizó el comando [0, 0, {...}] para generar una línea vinculada.

6. Cotización y pedido confirmado. La cotización es una propuesta de venta que todavía no ha sido confirmada. Tras ejecutar action_confirm, Odoo puede registrar el estado sale, correspondiente a una orden de venta confirmada. Confirmar no equivale a cobrar, facturar ni entregar los productos.

7. partner_id y order_line. partner_id identifica al contacto asociado al pedido mediante una relación many2one con res.partner. order_line agrupa las líneas mediante una relación one2many con sale.order.line. En el cuerpo JSON de creación se usan estos campos para asociar el cliente y los dos productos.

8. Identificadores de registros. Valores como partner_id = 6, productos 7 y 8 o el pedido ID 1 fueron observados en la base odoo20_integradores. En otra instalación pueden ser distintos. Antes de reutilizar una solicitud, se deben consultar los identificadores reales de la propia base de datos.

Uso pedagógico. Estos fundamentos ayudan a interpretar las pruebas documentadas, pero no sustituyen el diseño ni la implementación de la aplicación móvil y su API ASP.NET Core con Clean Architecture.

3. Verificación del entorno Docker

Objetivo: confirmar que Odoo y PostgreSQL están disponibles antes de ejecutar solicitudes.

```
cd /d D:\Odoo20-Docker
docker compose ps
```

Resultado observado: servicio db (postgres:16) en estado Up (healthy); servicio odoo (odoo:20.0) en estado Up; puerto publicado 127.0.0.1:8069→8069. No se reiniciaron contenedores.

4. Verificación de módulos instalados

```
docker compose exec db psql -U odoo -d odoo20_integradores -c "SELECT name, state FROM
ir_module_module WHERE name IN ('sale', 'product', 'point_of_sale') ORDER BY name;"
```

Módulo	Resultado real
point_of_sale	installed
product	installed
sale	installed

Esta consulta fue diagnóstica y de solo lectura. La aplicación ASP.NET Core no debe acceder directamente a PostgreSQL.

5. Preparación y verificación de los productos

Se comprobó la existencia del producto inicial ID 7 y se creó un segundo producto desde Ventas → Productos → Productos → Nuevo. Para el segundo se ingresaron: nombre Producto Prueba Integradores 2, referencia PROD-TEST-002, precio 15,00 USD, tipo Bienes, opción Ventas activada, rastreo de inventario desactivado.

El primer producto tenía referencia PROD-TEST-001 y precio 10,00 USD. Ambos quedaron activos y habilitados para ventas.

Consulta diagnóstica inicial:

```
docker compose exec db psql -U odoo -d odoo20_integradores -c "SELECT pp.id, pt.name,
pp.active, pt.sale_ok FROM product_product pp JOIN product_template pt ON pp.product_tmpl_id =
pt.id ORDER BY pp.id LIMIT 20;"
```

Consulta de verificación por referencia interna:

```
docker compose exec db psql -U odoo -d odoo20_integradores -c "SELECT pp.id, pp.default_code,
pt.name, pp.active, pt.sale_ok FROM product_product pp JOIN product_template pt ON
pp.product_tmpl_id = pt.id WHERE pp.default_code IN ('PROD-TEST-001', 'PROD-TEST-002') ORDER BY
pp.id;"
```

ID	Referencia	Activo	Puede venderse
7	PROD-TEST-001	Sí	Sí
8	PROD-TEST-002	Sí	Sí

Si psql muestra (END), presionar Q para salir del paginador.

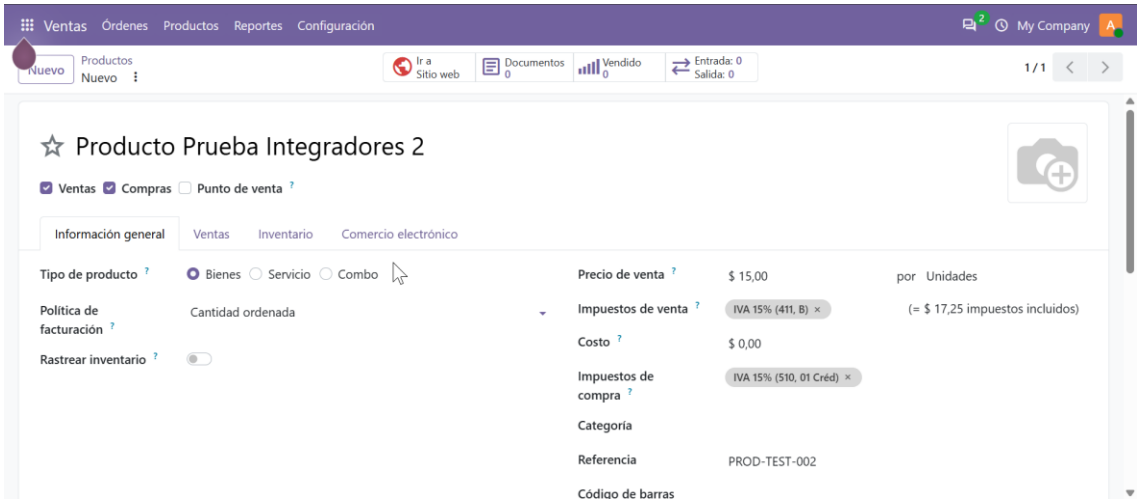


Figura. Ficha del segundo producto en Odoo.

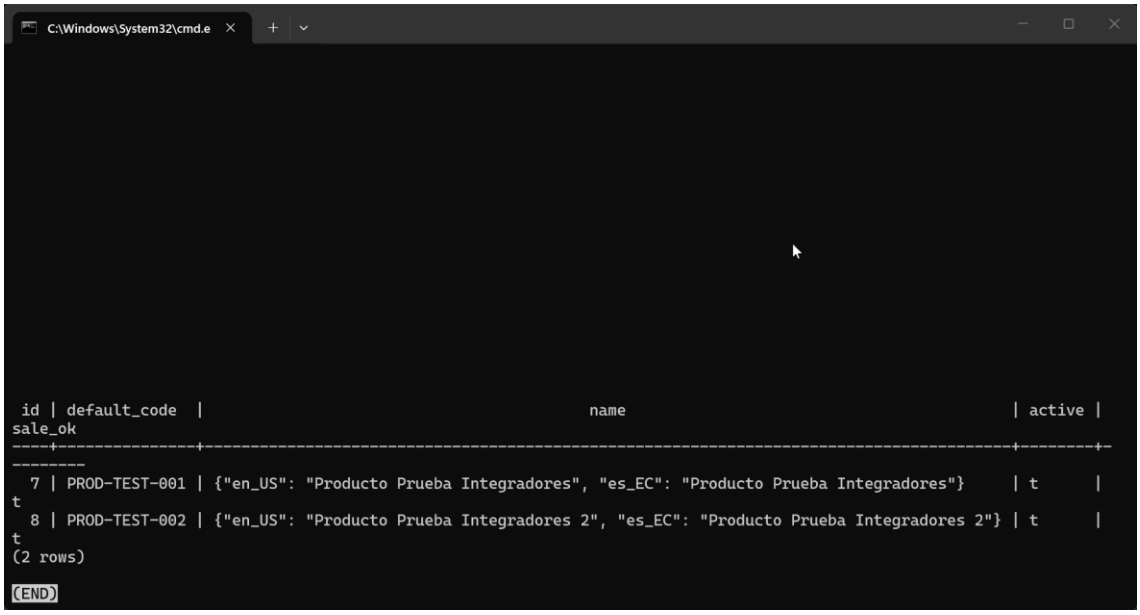


Figura. Comprobación de identificadores de los dos productos.

6. Verificación del contacto

```
docker compose exec db psql -P pager=off -U odoo -d odoo20_integradores -c "SELECT id, name, active, customer_rank FROM res_partner WHERE active = true ORDER BY id LIMIT 20;"
```

Resultado relevante: Consumidor final, ID 6, active=true, customer_rank=0. El valor 0 no impidió crear el pedido. El contacto ID 7, Instituto Ecuatoriano de Seguridad Social, se utilizó más tarde únicamente para probar el filtro de consulta.

7. Inspección de la documentación JSON-2

1. Abrir http://localhost:8069/doc en el navegador.
2. Buscar sale.order en Search for models y seleccionar Orden de venta (sale.order), no sale.order.line.

3. En Modules, pulsar Select all para mostrar métodos y campos de todos los módulos.
4. En Methods, localizar create; la ruta observada fue /json/2/sale.order/create, con parámetro vals_list de tipo list[ValuesType] y retorno list[int].
5. En Fields, comprobar partner_id (many2one a res.partner) y order_line (one2many a sale.order.line).

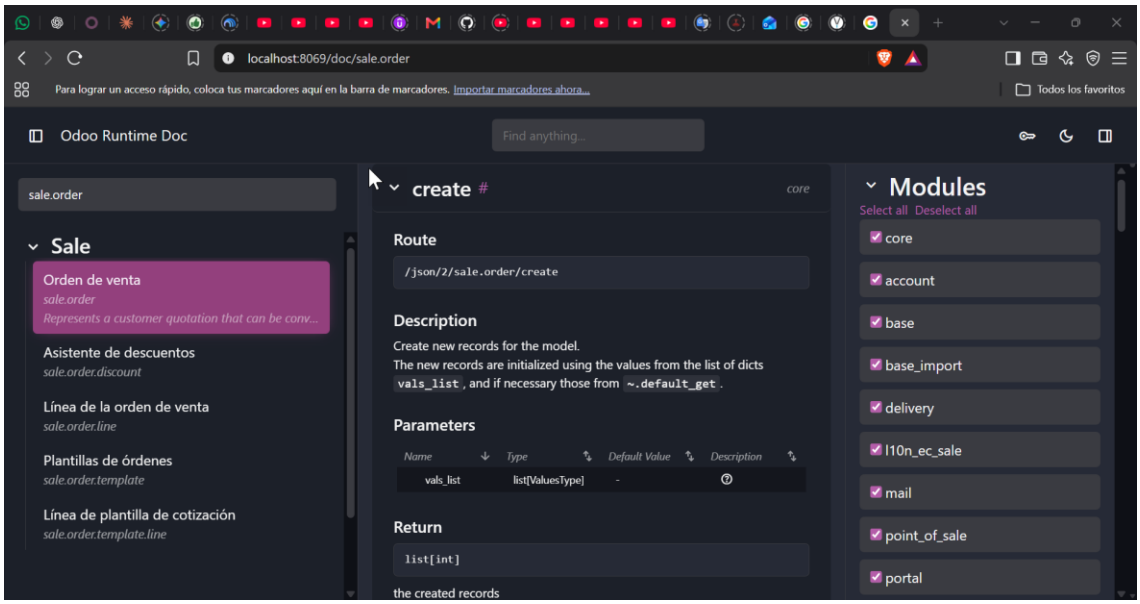


Figura. Método create documentado en sale.order.

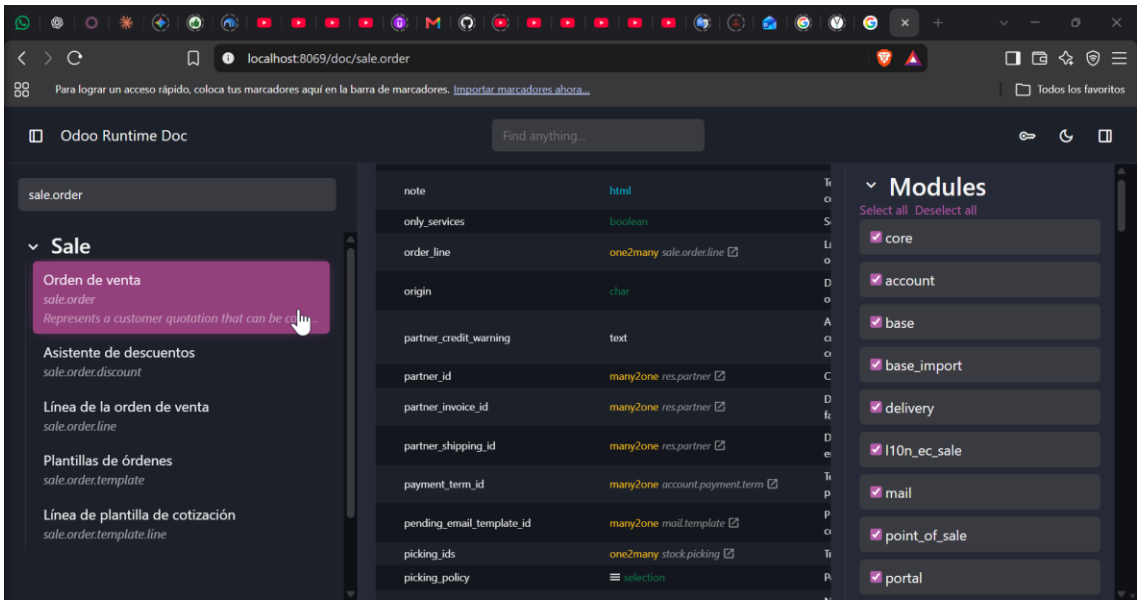


Figura. Campos partner_id y order_line del modelo sale.order.

8. Crear cotización desde Postman

Objetivo: crear mediante API JSON-2 una cotización con dos líneas sin escribir en PostgreSQL. Ejecutar esta solicitud una sola vez para evitar duplicados.

6. En Postman crear una solicitud nueva y seleccionar POST.
7. Introducir URL http://localhost:8069/json/2/sale.order/create.

8. En Authorization seleccionar Bearer Token e introducir únicamente la API Key; Postman añade Authorization: Bearer automáticamente.
9. En Headers agregar X-Odoo-Database = odoo20_integradores y Content-Type = application/json.
10. En Body elegir raw → JSON y pegar el contenido siguiente.

```
{
  "vals_list": [
    {
      "partner_id": 6,
      "client_order_ref": "PRUEBA-INTEGRADORES-001",
      "order_line": [
        [0, 0, {"product_id": 7, "product_uom_qty": 1, "price_unit": 10.0}],
        [0, 0, {"product_id": 8, "product_uom_qty": 2, "price_unit": 15.0}]
      ]
    }
  ]
}
```

Fundamento: vals_list es la lista de pedidos a crear; partner_id vincula al cliente; order_line contiene las líneas; el comando [0,0,{...}] crea un registro relacionado; product_uom_qty representa la cantidad; price_unit el precio unitario. El subtotal esperado es 1×10 + 2×15 = 40 USD, antes de IVA.

11. Pulsar Send una sola vez. En la sección inferior Response → Body revisar el estado HTTP y el JSON de respuesta. Si está contraída, arrastrar hacia arriba el separador horizontal.

Resultado real: HTTP 200 OK; cuerpo [1]. El identificador creado fue 1 (no es un valor fijo para futuras ejecuciones).

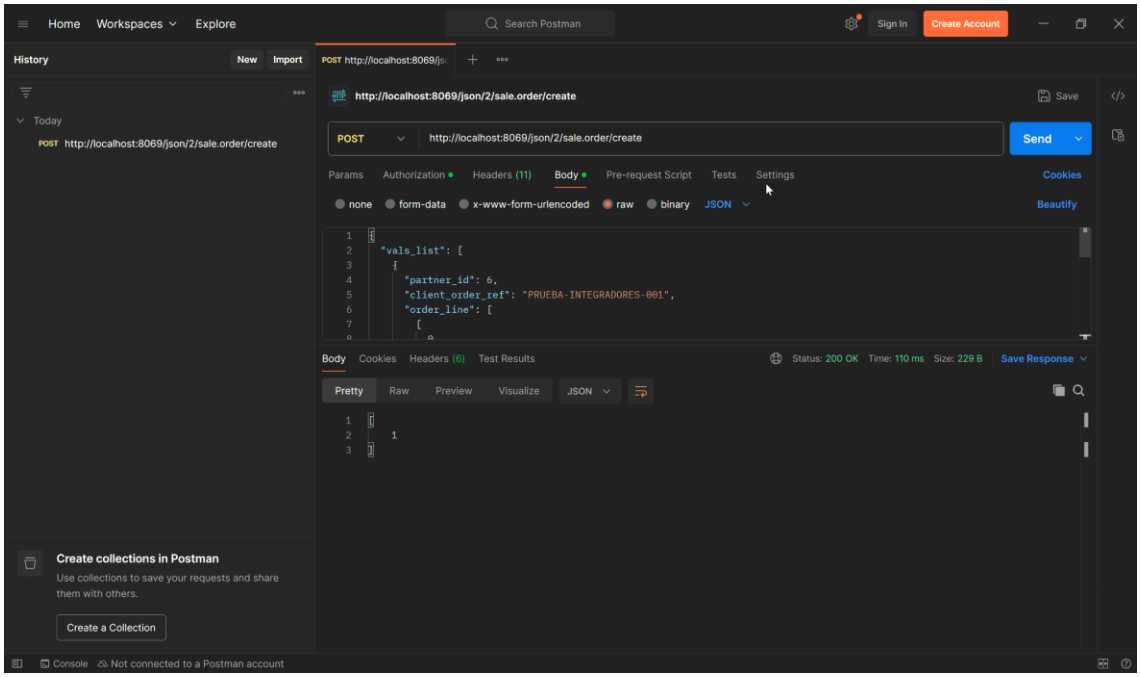


Figura. Postman devolvió HTTP 200 y el ID del pedido [1].

9. Verificar la cotización en Odoo

12. Abrir Ventas → Órdenes → Cotizaciones y localizar S00001 del cliente Consumidor final.
13. Abrir la cotización y verificar la referencia PRUEBA-INTEGRADORES-001 y las dos líneas de productos.

Línea	Cantidad	Precio unitario	Subtotal
PROD-TEST-001	1	10,00	10,00
PROD-TEST-002	2	15,00	30,00
Subtotal			40,00
IVA 15 %			6,00
Total			46,00

Resultado real: cotización S00001, cliente Consumidor final, subtotal 40 USD y total 46 USD. Esta operación creó una cotización, no un pago ni una venta confirmada.

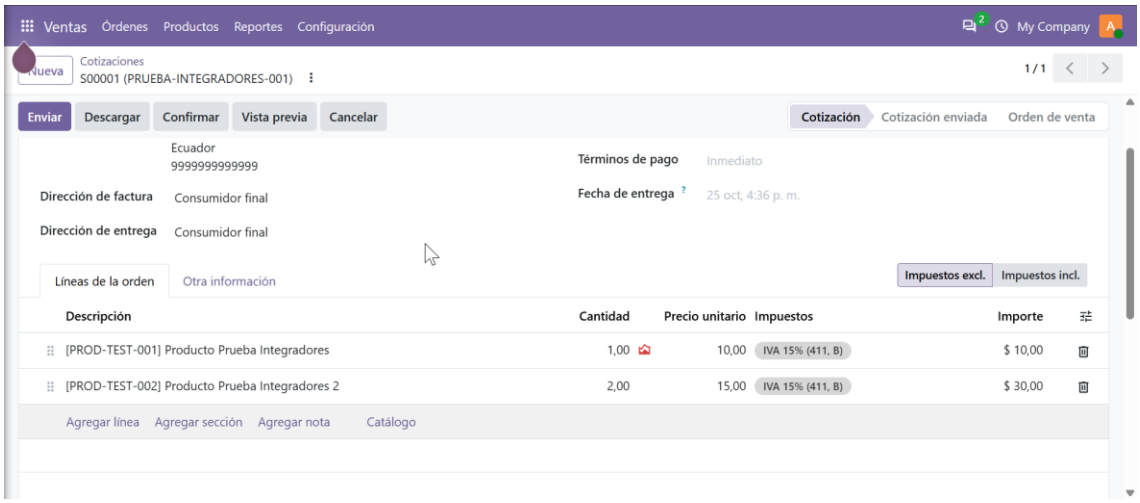


Figura. Detalle de la cotización S00001 con los dos productos.

10. Confirmar la cotización por JSON-2

Objetivo: verificar que el pedido puede pasar de cotización a orden de venta desde un cliente externo.

14. En Runtime Doc seleccionar específicamente el modelo sale.order y localizar el método action_confirm. Verificar que la ruta pertenezca a sale.order, no a otro modelo con un método homónimo.

Ruta observada: /json/2/sale.order/action_confirm; método sin parámetros propios; para operar sobre el registro se envía ids.

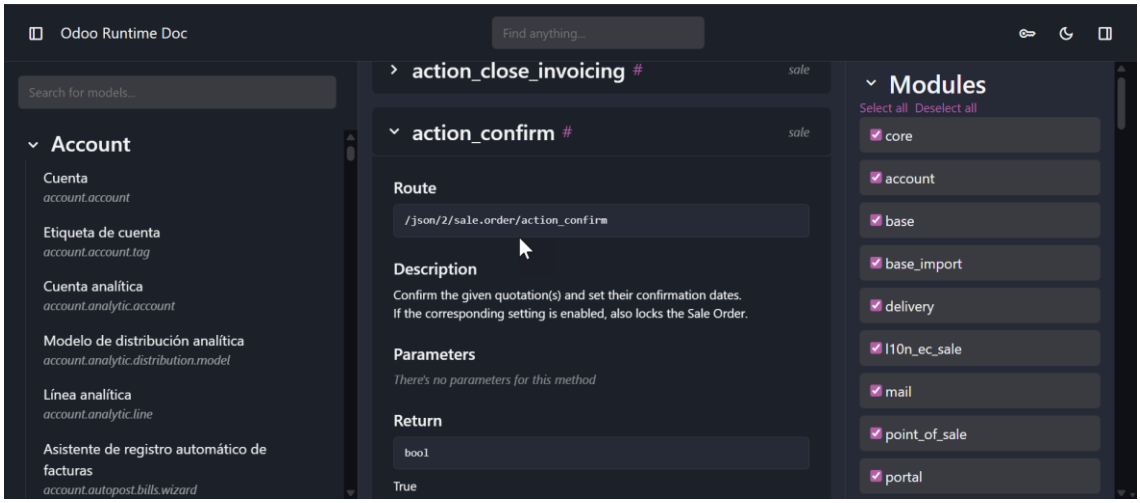


Figura. Método action_confirm correspondiente al modelo sale.order.

15. En Postman crear otra pestaña, seleccionar POST y URL http://localhost:8069/json/2/sale.order/action_confirm.
16. Configurar Bearer Token y las mismas cabeceras del paso de creación.
17. En Body → raw → JSON introducir el contenido siguiente y pulsar Send una sola vez.

```
{
  "ids": [1],
  "context": {}
}
```

Resultado real: HTTP 200 OK y respuesta true. La confirmación puede activar procesos logísticos de Odoo; no implica pago.

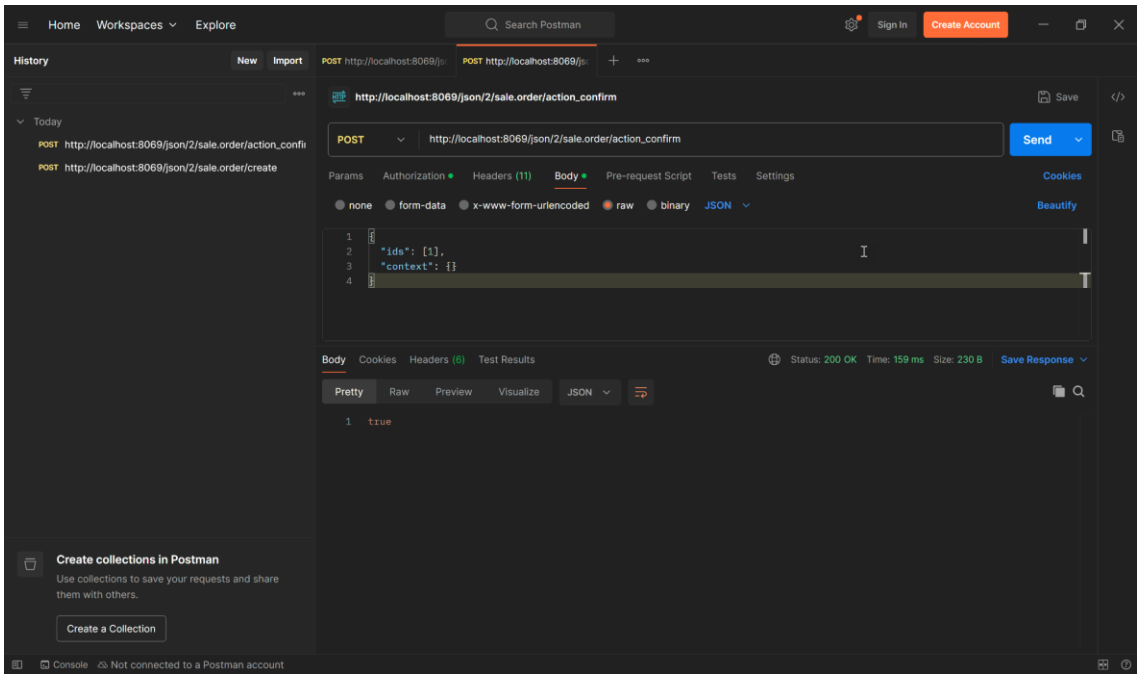


Figura. Respuesta true del método action_confirm en Postman.

18. En Odoo abrir Ventas → Órdenes → Órdenes de venta y localizar S00001. Verificar que el estado cambió a Orden de venta.

Resultado observado: orden de venta S00001, total 46 USD, estado de facturación Por facturar y entrega pendiente.

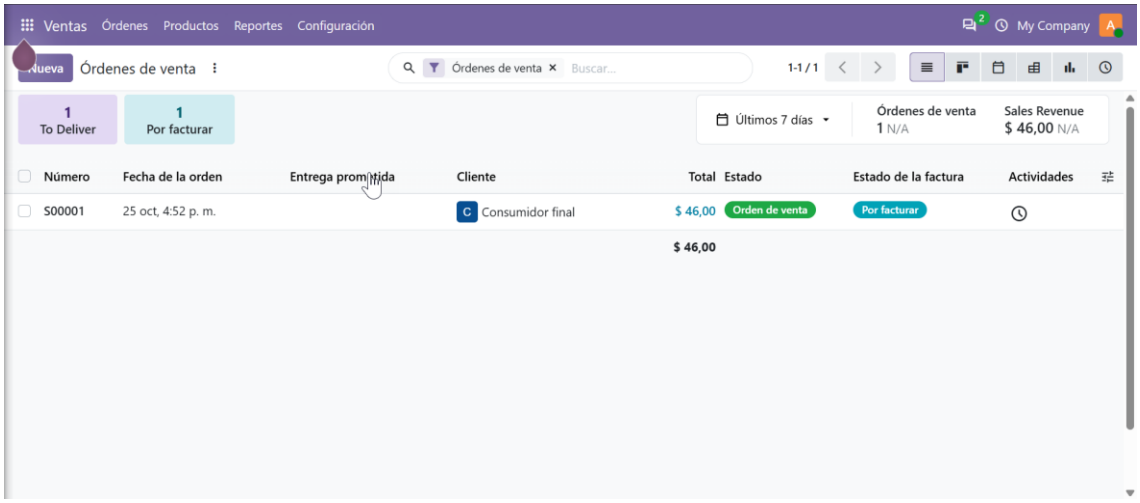


Figura. Orden S00001 confirmada y visible en la interfaz de Odoo.

11. Consultar el pedido confirmado

Objetivo: verificar lectura externa de estado, cliente, total y líneas de pedido. En JSON-2 las llamadas a métodos, incluidas las consultas, usan POST. Esto no significa que search_read modifique los datos.

19. Crear otra pestaña de Postman. Método POST, URL http://localhost:8069/json/2/sale.order/search_read, Bearer Token y las cabeceras habituales.
20. En Body → raw → JSON ingresar:

```
{
  "domain": [["id", "=", 1]],
  "fields": ["id", "name", "partner_id", "state", "amount_total", "order_line"],
  "limit": 1
}
```

21. Pulsar Send y revisar el resultado.

Respuesta real:

```
[
  {
    "id": 1,
    "name": "S00001",
    "partner_id": [6, "Consumidor final"],
    "state": "sale",
    "amount_total": 46.0,
    "order_line": [1, 2]
  }
]
```

Interpretación: state = sale identifica una orden de venta confirmada; order_line contiene los identificadores de sus dos líneas.

12. Comprobar el filtrado de pedidos por cliente

Objetivo: comprobar la capacidad de Odoo para devolver pedidos asociados a un partner_id. Esta prueba no equivale a validar controles de autorización de usuarios finales.

12.1 Cliente con pedido (partner_id = 6)

```
{
  "domain": [["partner_id", "=", 6]],
  "fields": ["id", "name", "partner_id", "state", "amount_total"],
  "limit": 20
}
```

Enviar a POST http://localhost:8069/json/2/sale.order/search_read, manteniendo Bearer Token y las cabeceras. Resultado real: un arreglo con el pedido ID 1, S00001, partner_id [6, "Consumidor final"], state "sale" y amount_total 46.0.

12.2 Otro contacto (partner_id = 7)

```
{
  "domain": [["partner_id", "=", 7]],
  "fields": ["id", "name", "partner_id", "state", "amount_total"],
  "limit": 20
}
```

Resultado real: [] (arreglo vacío). Por tanto, el pedido del cliente 6 no aparece en la consulta filtrada por cliente 7.

Límite de seguridad: la API Key utilizada tiene permisos de integración. Un cliente móvil no debe poder suministrar arbitrariamente partner_id para acceder a otros pedidos. El backend deberá autenticar y autorizar a los usuarios y derivar el identificador de cliente desde su identidad validada. No se implementó ni probó un mecanismo de autenticación de clientes en esta fase.

13. Incidencias y solución

Situación	Diagnóstico / acción
404 Not Found con mensaje Did you mean POST /json/2/<model>/<method>?	La consulta se había enviado con un verbo HTTP incorrecto; al cambiar a POST en search_read, la respuesta fue correcta.
En Runtime Doc se mostró delivery.note.wizard/action_confirm	Método homónimo de otro modelo; seleccionar sale.order y verificar la ruta completa antes de usarlo.
No se visualiza Response en Postman	Buscar la zona inferior, seleccionar Body y ampliar arrastrando el separador.
PostgreSQL muestra (END)	Presionar Q o usar psql -P pager=off.
Riesgo de pedidos duplicados	No repetir Send sobre create si ya devolvió el ID.

14. Matriz de evidencias y alcance del cierre

Operación	Evidencia real	Conclusión

Consulta de catálogo	Lectura de producto previamente verificada mediante API	Viable
Crear cotización	HTTP 200, [1], S00001 con 2 productos	Validada
Confirmar pedido	HTTP 200, true, estado Orden de venta	Validada
Consultar pedido	search_read devuelve S00001 y state sale	Validada
Filtrar por cliente	Cliente 6: S00001; cliente 7: []	Validada como filtro
Autenticación y autorización	No se ejecutaron pruebas de identidad de clientes	No validada
Pago y entrega	No se probaron transacciones de pago ni cierre logístico	Fuera de esta validación
App Flutter/Android y API .NET final	No se implementaron como parte de esta prueba	Trabajo estudiantil

15. Conclusión técnica

La instalación probada de Odoo 20 Community permitió crear, confirmar, consultar y filtrar pedidos sale.order mediante solicitudes JSON-2, sin modificar el código de Odoo y sin escrituras directas en PostgreSQL. Las pruebas respaldan la viabilidad de la integración básica del proyecto de Aplicaciones Móviles. No demuestran que la aplicación completa esté construida ni que la autenticación y autorización de clientes, pagos, entregas y demás funcionalidades estén validadas.